NIMBLEPROS

When Logs Tell the Whole Story

OpenTelemetry for Beginners

Barret Blake

#OTEL | #Observability | #OpenTelemetry

NIMBLEPROS

Barret Blake

Architect

- Microsoft MVP - .NET & Power Automate
- Husband & Father
- Coder
- Blogger & Speaker
- Mentor
- Gamer
- Buckeyes & Browns Fan
- LEGO Enthusiast
- Model Railroader

MVP Microsoft® Most Valuable Professional

NimblePros
Better software development. Delivered.

THE PROBLEM

Production failures are inevitable.

What happens next is up to you.

No observability

Guessing in the dark. Each incident is a mystery. MTTR spirals.

Scattered logs

Logs exist but tell disconnected stories. Hard to correlate across services.

Structured telemetry

Clear, connected story. Logs trace the path from failure to root cause.

The difference between a quick fix and a prolonged outage often comes down to whether your logs tell a clear, connected story.

FOUNDATION

What is OpenTelemetry?

An open-source, language-agnostic, vendor-neutral framework for application observability — providing APIs, libraries, services, and agents to capture traces, metrics, and logs.

1

Open Source

Backed by CNCF. Community driven.

2

Language Agnostic

.NET, Java, Go, JS, Python, Ruby, Rust & more

3

Vendor Neutral

Send your data to ANY compatible platform

Emerging industry standard for cloud-native observability

CORE CONCEPTS

The Four Signals of OTEL

Traces

Track the full path of a request through your system — from entry point to database and back.

Metrics

Numeric measurements over time — request counts, error rates, latency percentiles.

Logs

Timestamped text records that capture what happened and why. The base unit of all telemetry.

Baggage

Key-value context propagated across all spans and service boundaries within a trace.

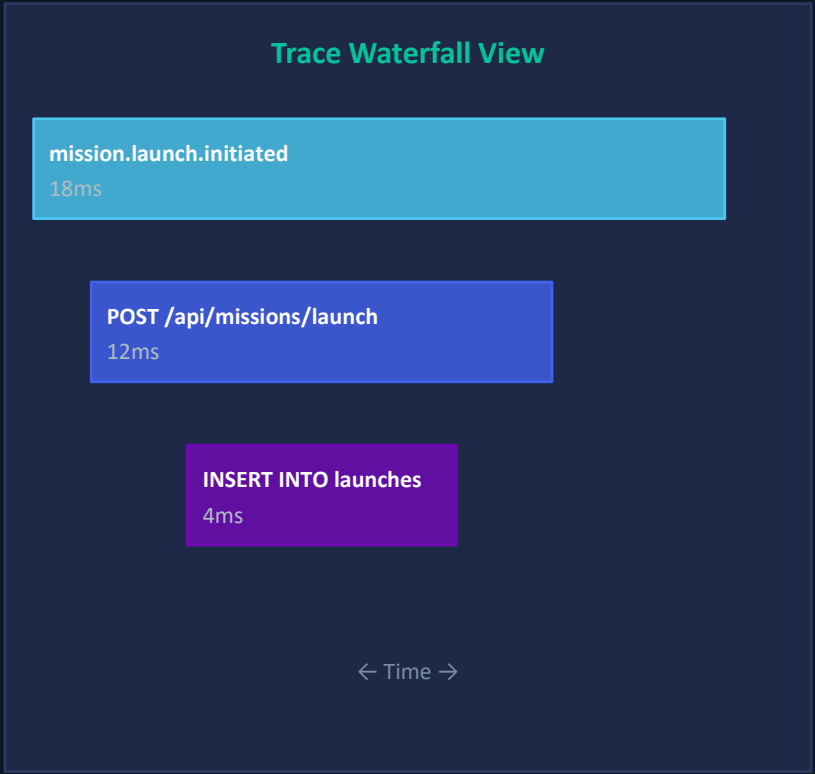
TRACES

Traces: Following the Request

A trace records the complete path of a request through your application.

- Each unit of work = a Span
- Spans nest into parent/child trees
- Captures timing for every step
- Spans across services share a TraceId
- Critical for cross-service debugging

Real-world impact: Analyzing trace waterfall charts on a client project revealed several performance bottlenecks — resulting in 10–40% runtime reduction across key workflows.



LOGS + TRACES

Trace Context: Connecting the Dots

The same TraceId and SpanId appear in both your logs AND your trace spans.

Log Entry
(with TraceId)

→

TraceId:
26bf82c5...

→

Trace Waterfall
(full request path)

One click to context

Jump from a log line directly to the full trace waterfall in your observability platform.

Cross-service correlation

The same TraceId flows through all services. Find log entries for a user's request across microservices instantly.

No manual correlation

OTEL propagates trace context automatically via HTTP headers and messaging metadata. Zero effort.

METRICS

Metrics: Measuring What Matters

🚨 Error Rate Alerting

Don't alert on every failure — you'll train your team to ignore alerts. Instead, use a metric counter and trigger alerts only when thresholds are exceeded (e.g., 20+ failures in 60 seconds).

📈 Code Usage Analysis

Before investing in a refactor, use metrics to prove it's worth the effort. If a 1,000-line class runs hundreds of times per minute, that data is your business case.

```
// Custom counter setup
var meter = new Meter(
  "Mission.Counter"
);
var counter = meter
  .CreateCounter<long>(
    "missions_launched"
);

// In your endpoint:
counter.Add(1, new
  KeyValuePair(
    "mission", missionName
  ));
```

Metric output: 3 launches for Enterprise, 1 launch for Voyager

CORE CONCEPTS

Baggage: Context That Travels

Baggage is key-value context that propagates automatically across all spans, services, and HTTP headers within a trace — accessible everywhere.

```
// Set baggage at request start
Baggage.SetBaggage(
  "mission.commander", "Picard"
);
Baggage.SetBaggage(
  "mission.priority", "High"
);
// Read anywhere in the trace
var commander = Baggage
  .GetBaggage("mission.commander");
```

✅ Good uses for Baggage

- User ID for correlation
- Tenant ID in multi-tenant apps
- Source IP address
- Feature flag group
- Request source/channel

⚠️ **Security Warning: Baggage is added to HTTP headers and is readable by any receiving service. Never put PII, passwords, or sensitive data in baggage.**

LOGS

Logs: The Story Behind the Numbers

> *"Not all logs are events, but all events are logs."* — *OpenTelemetry docs*

Automatically shipped

OTEL automatically exports log records you already create. No special wiring needed — the exporter handles it.

Structured output

OTEL transforms any log format into a structured record with defined fields: timestamp, severity, body, attributes.

Correlated with traces

Every log record within a trace carries the TraceId and SpanId — connecting logs to the request that triggered them.

When you need to understand what went wrong and why — logs are where you look.

LOGS

Anatomy of an OTEL Log Record

```
LogRecord.Timestamp:   2026-01-28...
LogRecord.TraceId:     26bf82c5...
LogRecord.SpanId:      c0da8620...
LogRecord.TraceFlags:  Recorded
LogRecord.CategoryName: Program
LogRecord.Severity:    Info
LogRecord.Body:        Mission launch
  requested: {missionName}...
LogRecord.Attributes:
    missionName: Enterprise
    status: Launched
```

Field	Purpose
Timestamp	When the event occurred
TraceId	Links log to its trace
SpanId	Specific span within trace
TraceFlags	W3C standard trace flags
CategoryName	.NET ILogger category
Severity	Log level (Info, Warn, Error...)
Body	The log message
Attributes	Structured key-value data

Source code: `logger.LogInformation("Mission launch requested: {missionName}, status: {status}", missionName, status);`

BEST PRACTICES

Choosing the Right Log Level

Level	Use When…	Recommended For
Trace	Highly detailed debugging info. Very high volume. Dev only.	*Never in production*
Debug	Diagnostic details for development and debugging.	*Dev / staging only*
Information	Normal operations. Application milestones, key events.	*Production — selectively*
Warning	Unexpected but recoverable situations. Worth monitoring.	*Always in production*
Error	Operation failed. Requires attention but app still running.	*Always in production*
Critical	Catastrophic failure. Immediate action required.	*Always + alerting*

ANTI-PATTERNS

Common Logging Mistakes to Avoid

🌊 Over-Logging

Problem: Logging everything creates noise that buries the signal. High-volume logging also drives up observability costs on commercial platforms.

✅ Fix: Log meaningful events. Use appropriate levels. Filter dev-only logs from production.

🔤 Unstructured Messages

Problem: Plain string concatenation ('Error for user ' + id) can't be queried. You can't filter, aggregate, or alert on it.

✅ Fix: Use structured logging: LogError("User {UserId} failed checkout", userId) — OTEL captures it as a queryable attribute.

🔕 Missing Context

Problem: A log that says 'Operation failed' is useless without knowing WHICH operation, for WHICH user, with WHAT data.

✅ Fix: Always include relevant identifiers: user IDs, record IDs, operation names. Failures are almost always data-specific.

🔔 Alert Fatigue Loops

Problem: Alerting on every single failure trains your team to create inbox rules to ignore alerts — defeating the purpose entirely.

✅ Fix: Use metrics with thresholds, not per-event alerts. Alert on patterns, not individual occurrences.

GETTING STARTED

Setting Up OTEL in .NET

Step 1: Add NuGet Packages

- `OpenTelemetry.Extensions.Hosting`
- `OpenTelemetry.Instrumentation.AspNetCore`
- `OpenTelemetry.Exporter.Console`

That's it for basic setup!

OTEL auto-instruments your ASP.NET Core routes and starts collecting logs, traces, and metrics immediately.

Step 2: Configure in Program.cs

```
builder.Logging
  .AddOpenTelemetry(opts => {
    opts.SetResourceBuilder(
      ResourceBuilder
        .CreateDefault()
        .AddService("app-logger"))
    .AddConsoleExporter();
  });

builder.Services
  .AddOpenTelemetry()
  .ConfigureResource(r =>
    r.AddService("app-logger"))
  .WithTracing(t => t
    .AddAspNetCoreInstrumentation()
    .AddConsoleExporter())
```

For Aspire projects: OTEL is included out of the box in the ServiceDefaults template — no manual setup required.

PRODUCTION

Exporting to Production Platforms

Self-Hosted

- **Jaeger**
  Distributed tracing platform
- **Grafana + Loki**
  Metrics, logs, and dashboards
- **Elastic Stack**
  Search-first observability

Pros: You control costs, data ownership
Cons: You own maintenance + updates

Commercial SaaS

- **Datadog**
  Full-stack observability
- **Grafana Cloud**
  Hosted Grafana + alerting
- **Azure App Insights**
  Native .NET + Azure integration
- **New Relic**
  AI-powered incident response

Pros: Managed, enterprise features
Cons: Cost scales with data volume

The OTEL advantage: switching platforms requires only a config change. Your data model stays the same.

TOOLING

Aspire: OTEL Built In for Local Dev

Console View

Raw log output across all services in one pane

Structured Logs

Filter and search by severity, category, or attribute

Trace List

Browse all traces, filter by service or duration

Waterfall View

Visual span timing — pinpoint slow code instantly

Metrics

Runtime metrics, custom counters, all in one dashboard

Multi-Language

Aspire v13+ supports JS, Python, Node.js and more

⚠️ Don't deploy the Aspire dashboard to production — it exposes full debug logs with no auth. Use a dedicated observability platform in deployed environments.

ADVANCED PATTERNS

Reusable Telemetry with Aspire ServiceDefaults

The ServiceDefaults shared project is the perfect home for reusable telemetry patterns across your entire Aspire solution.

Custom Metric Counters

Define counters in ServiceDefaults via AddMeter extension, then call counter.Add(1) in any endpoint. Metrics appear instantly in the Aspire dashboard.

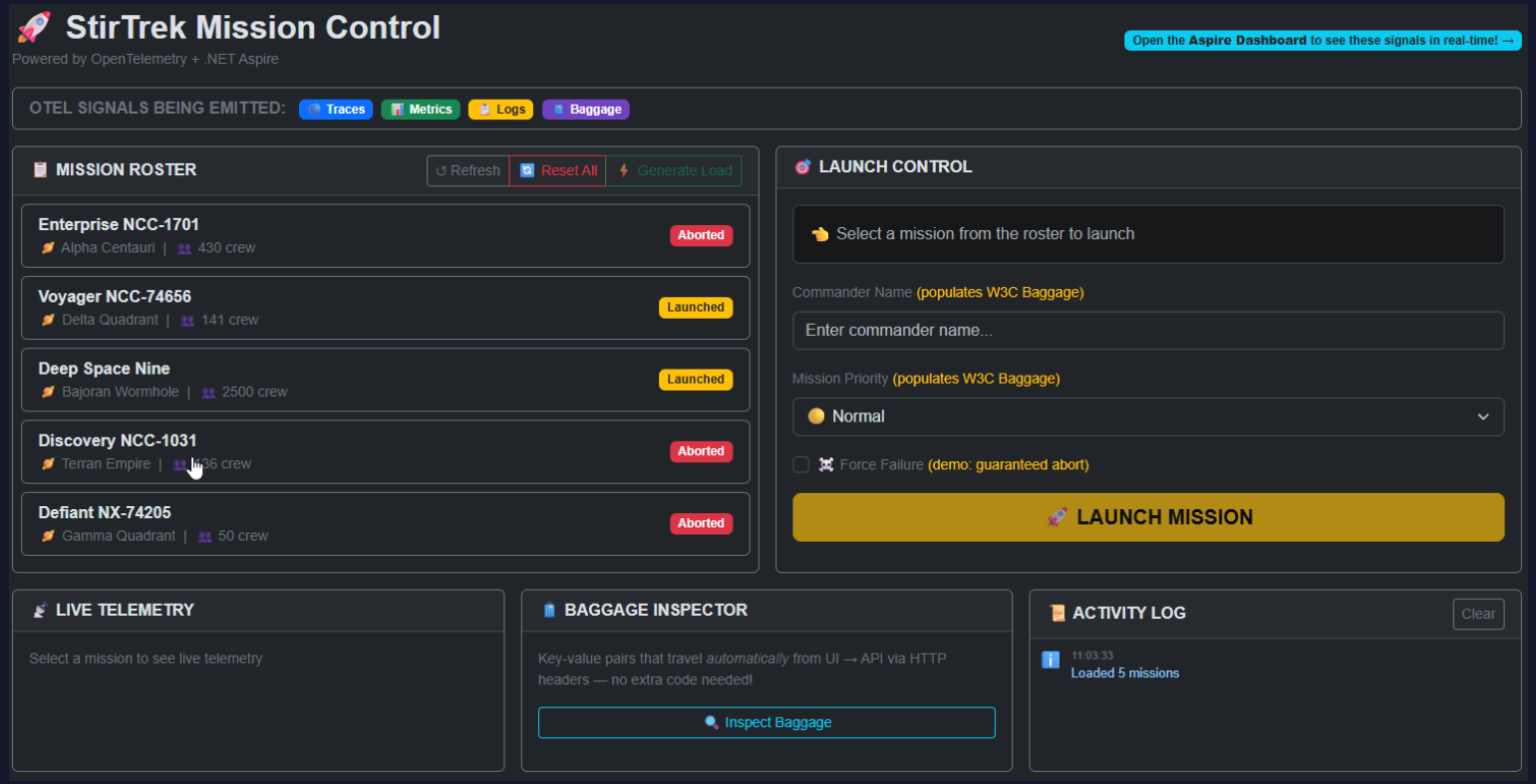
Reusable Span Factory

Create generic GetDatabaseSpan<T>() in ServiceDefaults — auto-tags span attributes from your entity's fields. One method, consistent tracing everywhere.

```
// ServiceDefaults: reusable span factory
public static Activity? GetDatabaseSpan<T>(
  this ActivitySource src, string name, T dbObj) {
  var activity = src.StartActivity(name, ActivityKind.Internal);
  // auto-tag all fields from entity…
  return activity;
}
```

Demo

DEMO TIME



SUMMARY

The Observability Stack in Action

1	**Incident Alert**	Metric threshold exceeded: 20+ errors in 60 seconds
2	**Find the Logs**	Filter by time window + service. Severity: Error
3	**Read Trace Context**	TraceId in log → jump to full trace waterfall
4	**Identify Root Cause**	Slow span or exception revealed in trace + attributes
5	**Fix & Verify**	Deploy fix. Watch metrics normalize. Incident closed.

Key Takeaways

- OpenTelemetry is the emerging standard — vendor-neutral, language-agnostic observability for modern apps.
- Logs + Traces + Metrics work together. TraceId connects log entries to full request paths across services.
- Log level discipline matters. Use structured messages with meaningful attributes — not noisy string concatenation.
- Baggage propagates context automatically. Great for tenant ID, user ID, feature flags — never PII.
- Avoid over-logging and alert fatigue. Use metric thresholds to alert on patterns, not individual events.
- Aspire's ServiceDefaults makes telemetry reusable. Define once, instrument everywhere — including custom spans and counters.
- OTEL gives you freedom to switch platforms. Your observability investment is portable — a config change, not a rewrite.

Thank You!

Questions & Discussion

Barret Blake

Architect, NimblePros

Microsoft® Most Valuable Professional

NimblePros
Better software development. Delivered.

Resources

- opentelemetry.io/docs/what-is-opentelemetry/
- aspire.dev/fundamentals/telemetry/
- learn.microsoft.com — .NET observability with OpenTelemetry
- github.com/barretb/stirtrekdemo2026

Contact Me

- linkedin.com/in/barretblake
- bluesky: @barretblake.dev
- barretblake.dev
- youtube.com/@barretcodes